GPU-Accelerated Martingale Posterior Neural Processes for MNIST Image Completion

Jose Perales
EN605.617: Introduction to GPU Programming
Johns Hopkins University

Spring 2026

Abstract

The Martingale Posterior Neural Process (MPNP, Lee et al., ICLR 2023) replaces the arbitrary latent prior of a standard Neural Process with predictive resampling. At every training step the model draws synthetic context sets from its own predictive distribution and refits. The payoff is calibrated, prior-free uncertainty over functions. An MPNP provides calibrated doubt, not just inference, the kind a deployed model needs before it brakes for a pedestrian, flags a tumor, or sizes a tail-risk position. The cost is that every gradient step now runs many encoder and decoder forward passes per task, making the model GPU-bound.

This paper takes a working PyTorch MPNP implementation for MNIST image completion and uses it as a vehicle for a tour of the CUDA stack. The bulk of the paper walks through which NVIDIA library kernel each MPNP op actually launches, such as cuBLAS, cuDNN, SDPA, and the elementwise/reduction kernels behind the loss. Then, this paper catalogues the PyTorch knobs that move the needle around them: the ATen dispatcher, the caching allocator, pinned memory, AMP and TF32, `cudnn.benchmark`, CUDA Graphs, `torch.compile`, and the profiler workflow. Each is paired with its documented behavior, its gotcha, and the specific MPNP code change it implies.

1 Introduction

Models that return only a point estimate are often unsuitable in risky environments. Autonomous driving needs a confidence band on the pedestrian-detection score before it commits to braking. In clinical decision support, a model that flags its own uncertainty is more useful than one that is confidently wrong. Quantitative finance needs tail risk that is calibrated, not just estimated. Uncertainty quantification is critical wherever predictions inform consequential actions.

Lee, Yun, Nam, Fong, and Lee provide a solution to uncertainty quantification with the Martingale Posterior Neural Process (MPNP) [1], a Bayesian-style Neural Process that does not require a prior. Following Fong, Holmes, and Walker [2], they replace the prior with predictive resampling. At every step, the model generates K synthetic context sets from its own predictive distribution, refits, and uses the spread of refits as the posterior. Posterior updates form a martingale (expected belief over self-generated data does not drift). The result is coherent uncertainty without ever specifying a prior.

However, predictive resampling comes at a computational cost. The new objective requires pseudo-context forward passes through the encoder, decoder, and per-target Gaussian negative log-likelihood (NLL) per task per training step. This is a natural fit for the GPU, and it makes MPNP a useful case study in GPU programming. This paper traces how a PyTorch implementation

maps onto the CUDA stack underneath: dispatch, kernel libraries, memory, and the precision and graph-capture knobs around them.

2 Math Review

2.1 Martingale Posterior Distributions

Classical Bayesian inference asks "what do I believe about the parameter θ?"; a question that demands a prior $\pi(\theta)$ before any data is observed. Fong, Holmes, and Walker flip the question: "how do I expect future observations to look, given what data I already have?" That requires only a one-step predictive $P_n(\cdot \mid y_{1:n})$, no prior [2].

Uncertainty about θ becomes uncertainty about the missing future. Fong et al. design a predictive resampling mechanism to forward-simulate a long imaginary future from the current predictive, refit, and read off θ:

$$Y_i \sim P_{i-1}, \quad i = n{+}1, \ldots, N \quad \Longrightarrow \quad \theta_N = \theta(P_N) \sim \Pi_N(\cdot \mid y_{1:n}). \tag{1}$$

Repeating the simulation many times yields a full posterior Π_N over θ. The predictive sequence $\{P_n\}$ is a martingale, $\mathbb{E}[P_{n+1} \mid P_n] = P_n$, so beliefs are consistent and do not drift on self-generated data. There is no prior to choose and no likelihood to specify, only a predictive model and the samples it generates.

2.2 Neural Processes

The two classical answers to this problem space (predictions with calibrated uncertainty) each have a flaw. Neural networks are flexible, scalable function approximators but return only point predictions with no principled uncertainty. Gaussian processes give exact posterior mean and variance per query but cost $\mathcal{O}(n^3)$ per task and do not amortize across datasets.

Neural Processes (NPs) [3] sit in the middle. Like a Gaussian Process, an NP learns a distribution over functions from a context set $\mathcal{C} = \{(x_i, y_i)\}_{i=1}^n$, but it does so with a neural network and amortizes inference into a single forward pass:

$$p(y_t \mid x_t, \mathcal{C}) = \int p(y_t \mid x_t, z)\, q(z \mid \mathcal{C})\, dz. \tag{2}$$

A permutation-invariant encoder $q_\phi(z \mid \mathcal{C})$ (MLP plus mean-pool) maps the context set to a latent distribution, and a decoder $p_\theta(y_t \mid x_t, z)$ predicts each target from the latent. The model is trained by maximizing an evidence lower bound. The remaining problem is the same one Bayesian inference always has. The latent prior $p(z)$ is arbitrary, so the posterior is rarely calibrated.

2.3 Martingale Posterior Neural Processes

Lee et al. plug Fong's predictive resampling inside the NP training loop [1]. The model uses the same encoder and decoder, plus a pseudo-context generator, an Induced Set Attention Block (ISAB) [4], that samples K synthetic context sets $\mathcal{Z}_0^{(k)}$ from the model's own current predictive. This is a neural analogue of predictive resampling.

The training objective combines three terms:

$$\boxed{\mathcal{L}_{\text{MPNP}} = \underbrace{\mathcal{L}_{\text{amort}}}_{\text{predict from real } \mathcal{C}} + \underbrace{\mathcal{L}_{\text{marg}}}_{\substack{\text{log-mean-exp over } K \\ \text{pseudo-augmentations}}} + 0.1 \cdot \underbrace{\mathcal{L}_{\text{pseudo}}}_{\substack{\text{pseudo}\to\text{target} \\ \text{consistency}}} .} \tag{3}$$

Here

$$\mathcal{L}_{\text{amort}} = -\mathbb{E}_{q(z|\mathcal{C})}\left[\log p(y_T \mid x_T, z)\right], \tag{4}$$

$$\mathcal{L}_{\text{marg}} = -\log\left(\frac{1}{K}\sum_{k=1}^{K} p(y_T \mid x_T, \theta_k)\right), \qquad \theta_k = \theta\left(\mathcal{C} \cup \mathcal{Z}_0^{(k)}\right), \tag{5}$$

and $\mathcal{L}_{\text{pseudo}}$ scores the predictive quality of the pseudo-set alone.

$\mathcal{L}_{\text{marg}}$ requires K pseudo-augmented forward passes per task per step. With $B = 64$, $T = 784$, hidden width $h = 256$, and $K = 5$, the inner forward graph holds $\mathcal{O}(B \cdot K \cdot T \cdot h)$ activations and launches at least $K+1$ separate sequences of linear-layer, decode, and log-prob kernels per step. This is the bottleneck that the rest of the paper targets.

3 MNIST Example

3.1 Task Formulation

Each 28×28 MNIST image is flattened into 784 coordinate-intensity pairs:

- $x_i \in [-1, 1]^2$: normalized pixel coordinate (row, column).
- $y_i \in [0, 1]$: pixel intensity.

Per training task, a random subset of $n \in [10, 150]$ pixels is revealed as context. The model must predict all 784 pixel intensities and their per-pixel standard deviations. MNIST is a fitting test bed for a Neural Process because it has variable-size context, genuine spatial structure, and uncertainty that should localize on unobserved pixels and ambiguous edges. The original MPNP paper uses $n \in [10, 200]$. This implementation caps at 150 so a larger fraction of pixels is used as targets per task.

3.2 Architecture and Hyperparameters

The MPNP implementation reuses the encoder/decoder MLPs from `neural_process.py`, adapted from Dupont's NP repository [5]. The pseudo-context generator is a single ISAB block built from two `MAB` (multihead-attention block) layers with 8 heads.

The shapes that appear throughout Section 4 are listed in Table 1.

Tensor	Shape	Notes
context (x, y)	$(B, n, 3)$	n varies per batch
target x_T	$(B, 784, 2)$	fixed
target y_T	$(B, 784, 1)$	fixed
pseudo-context	$(B, K, M, 3)$	K=5, M=30
latent z	(B, K, z_{dim})	z_{dim}=256

Table 1: Tensor shapes used by the MPNP training step.

Hyperparameters from `train_inpainting.py` are listed in Table 2.

Hyperparameter	Value
Hidden width $h = r = z$	256
Pseudo-points M	30
Pseudo-samples K	5
Loss weights	$(\mathcal{L}_{\text{amort}}, \mathcal{L}_{\text{marg}}, \mathcal{L}_{\text{pseudo}}) = (1.0, 1.0, 0.1)$
Optimizer	AdamW, lr $5{\times}10^{-4}$, weight decay 10^{-5}
LR schedule	Cosine, $T_{\max} = 100$
Batch size	64
Epochs	100
Gradient clip	$\|g\|_2 \leq 5$
Train / val / test split	50 000 / 10 000 / 10 000

Table 2: Training hyperparameters.

3.3 Qualitative Results

After 100 epochs the model reaches a test NLL of -1.853; lower is better, and for a Gaussian over $[0, 1]$ pixels this corresponds to σ in the low tens of percent on average.

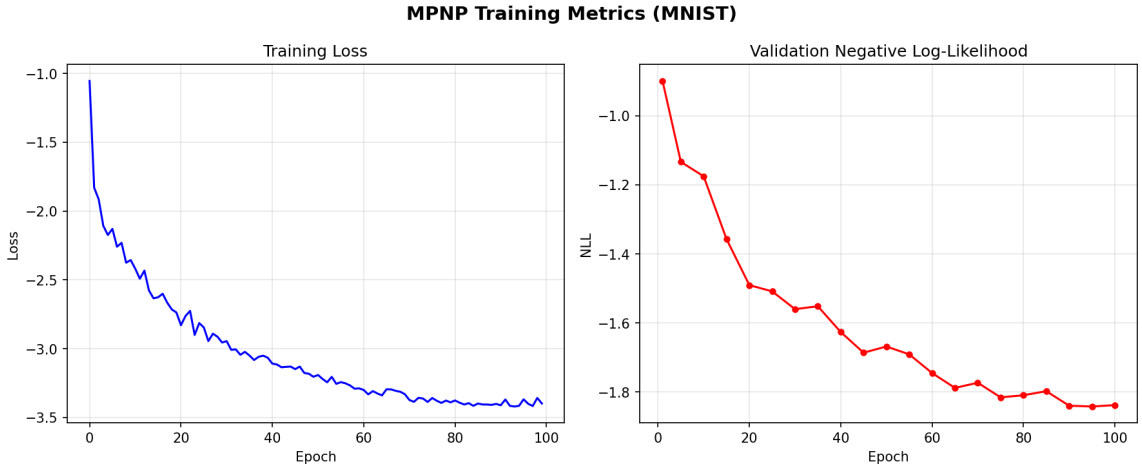

Figure 1: Training loss and validation NLL across 100 epochs. Loss decreases smoothly; validation NLL tracks training loss with no prior-collapse pathology and no overfitting.

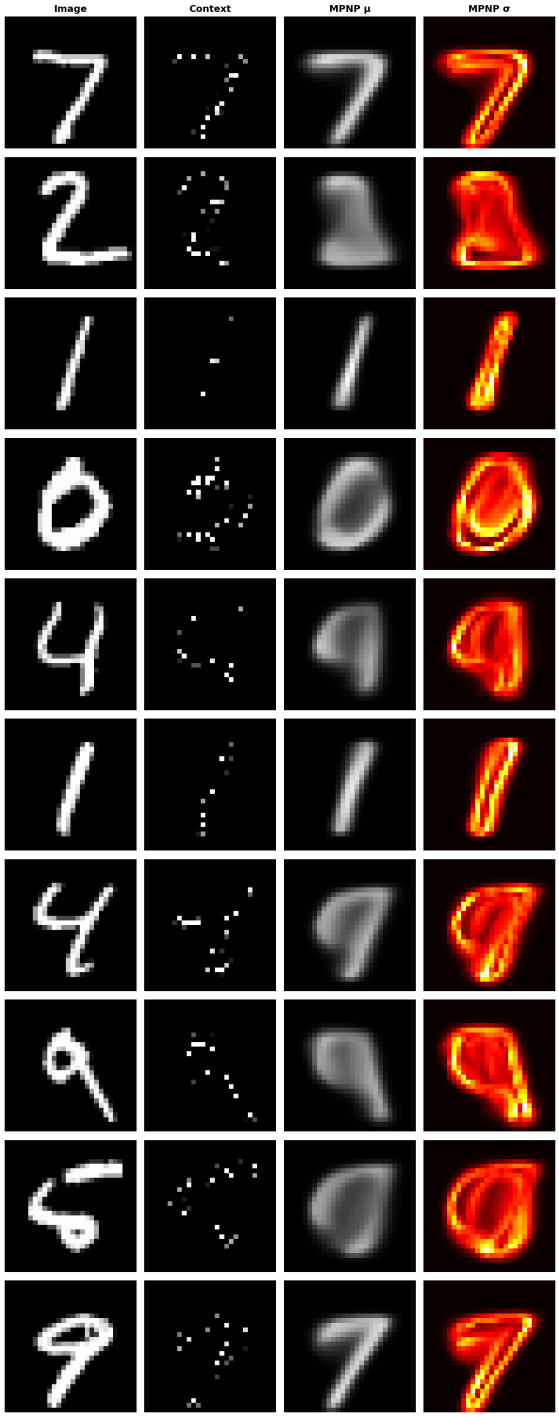

Figure 2: Test-set image completion. Columns: original digit, observed context pixels, predicted mean μ, predicted standard deviation σ. Uncertainty concentrates on unobserved regions and on ambiguous strokes, matching the calibration behavior reported by the MPNP paper.

The remainder of the paper analyzes how this training loop runs on the GPU.

4 CUDA Programming inside PyTorch

The earlier modules of this course built CUDA programs by hand: allocating global and pinned host memory, launching kernels across streams, calling cuBLAS and cuDNN directly, and reducing arrays with Thrust. The MPNP implementation in `train_inpainting.py` and `martingale_posterior_neural_process.` does none of that explicitly — yet a single training step issues dozens of CUDA kernel launches across cuBLAS, cuDNN, the caching allocator, and a handful of hand-written elementwise kernels. The purpose of this section is to make the implicit stack visible.

The section is organized around three questions, each mapped to a subsection. First, where in this project does the GPU actually do work? Section 4.1 inventories the GPU-touching call sites in the project's own files. Second, how does PyTorch make the GPU run those operations? Section 4.2 walks each backend (cuBLAS, cuDNN, TensorIterator, reductions, SDPA, the caching allocator) and names the hardware feature it exploits. Third, how does that connect to the hand-written CUDA we wrote in earlier modules? Section 4.3 draws those parallels module by module. Section 4.4 closes with the optimizations we would pull next, and Section 4.5 describes the profiler harness that would verify them.

4.1 Where the GPU runs in this project

This subsection answers the first rubric question: which lines of `train_inpainting.py` and `martingale_posterior_neural_process.py` actually exercise the GPU? The answer is, almost all of them, but in three distinct phases: the data path, the forward path, and the loss-and-backward path.

Data path. The MNIST point cloud is built on the CPU. `MNISTPointCloud.__init__` precomputes a 784×2 coordinate grid as a CPU tensor; `__getitem__` returns a CPU `(coords, values)` pair. The DataLoader is constructed with `pin_memory=True`, which routes those CPU batches through page-locked host memory so the subsequent `x_full.to(device)` in `train_epoch` can issue an asynchronous `cudaMemcpyAsync` instead of a blocking copy [6]. The synthetic `torch.randperm(n, device=x.device)` call inside `context_target_split` runs the permutation on the GPU using the cuRAND philox generator that backs PyTorch's CUDA RNG.

Forward path. A single training step calls `model.compute_loss(x_ctx, y_ctx, x_tgt, y_tgt)`, which in turn invokes `encode_set` once on the real context, then K=5 more times on each combined context, plus `decoder` calls in matching number. Each of those calls is a stack of `nn.Linear` layers separated by `ReLU` activations; each `nn.Linear` dispatches to `aten::addmm`, which is a cuBLAS GEMM with bias fusion. The mean-pool aggregator inside `encode_set` is a single `aten::mean` reduction. The hand-written `MultiHeadAttention` block inside the `ISAB` pseudo-context generator issues two `aten::matmul` kernels, an `aten::softmax`, and several small reshapes. The reparameterization $z = \mu + \sigma \odot \epsilon$ inside `q_context.rsample()` is one elementwise kernel scheduled through `TensorIterator`.

Loss and backward path. `Normal.log_prob(y_tgt)` expands into a sequence of elementwise kernels (`log`, `pow`, `sub`, `div`) followed by an `aten::mean` reduction; this fires K+1 times per step. The marginal-likelihood term in `compute_loss` stacks K such log-likelihoods and reduces them with a hand-rolled log-sum-exp (`max` → `sub` → `exp` → `mean` → `log`, four kernels where `torch.logsumexp` would be one). Calling `loss.backward()` traverses the autograd graph in reverse and launches the gradient kernel for each forward op above on the same CUDA stream the forward used [6]. The

`optimizer.step()` call on `torch.optim.AdamW` fires a fused multi-tensor elementwise kernel that updates every parameter in one launch.

Table 3 summarizes the mapping. The shapes use the project's actual hyperparameters: batch size B=64, hidden width h=256, context size $n \in [10, 150]$, target size T=784 pixels per image, latent width z=256, M=30 pseudo-context points, 8 attention heads, and K=5 pseudo-context samples.

MPNP step	ATen op	Backend / kernel	Tensor shape (this project)	Why GPU wins
Encoder MLP (`Linear` + `ReLU`)	`aten::addmm`	cuBLAS GEMM [13]	$(B{\cdot}n, 256) \times (256, 256)$	Tensor Cores [15]
Mean-pool over context	`aten::mean`	cooperative-groups reduction [17]	$(B, n, 256) \to (B, 256)$	parallel reduction
ISAB pseudo-context attention	`aten::matmul` + `aten::softmax`	cuBLAS GEMM + cuDNN softmax [14]	$(B, 8, M, 32)$	cuDNN-tuned softmax
Reparameterization $z = \mu + \sigma\epsilon$	`aten::add`, `aten::mul`	elementwise CUDA via `TensorIterator` [7]	$(B, 256)$	coalesced loads
Decoder MLP	`aten::addmm`	cuBLAS GEMM [13]	$(B{\cdot}T, 256) \times (256, 256)$	Tensor Cores
Per-pixel Gaussian NLL	`aten::log`, `aten::pow`, `aten::sub`	elementwise CUDA	$(B, 784, 1)$	parallel
Manual log-sum-exp over K	`aten::max`, `aten::exp`, `aten::log`	4 elementwise + reduction kernels	(K, B)	stable fp32 reduction
AdamW step	`aten::_foreach_*`	fused multi-tensor elementwise	all parameters	one launch per step

Table 3: Mapping from MPNP operations in this project to PyTorch ops, CUDA backend kernels, and the actual tensor shapes produced by the reference hyperparameters.

Two structural facts about this table matter for the rest of the section. First, the encoder, decoder, and per-pixel log-likelihood each fire K+2 times per training step — once for the real context pass and K more times inside the pseudo-context loop — so any per-launch cost is multiplied accordingly. Second, the only ops whose tensor shapes vary batch-to-batch are the encoder GEMM and the mean-pool reduction (both because n varies); every other shape is fixed by the architecture. Section 4.4 returns to both points.

4.2 How PyTorch uses the GPU

This subsection answers the second rubric question. Each backend named in Table 3 sits behind a small number of abstractions. Below, one short paragraph per backend names the library, cites its primary documentation, and identifies the hardware feature it exploits. The four-layer dispatch path (Python entry point → ATen device dispatch → `AT_DISPATCH_*` dtype switch → backend) is summarized at the end of the subsection.

cuBLAS / cuBLASLt. Every `nn.Linear` in `encode_set` and `decoder` dispatches to `aten::addmm`, which calls the cuBLAS `Sgemm` or `HgemmStridedBatched` routine [13]. On Ampere and later, those GEMMs run on Tensor Cores when the inputs are fp16 or bf16 (and on TF32 Tensor Cores when the inputs are fp32 and `torch.set_float32_matmul_precision("high")` is set) [15]. Tensor Cores

execute a $4 \times 4 \times 4$ matrix multiply per warp instruction, so a single MMA delivers the arithmetic of 64 fused multiply-adds; for the 256×256 MLP weights in this project the cost per layer is microseconds.

cuDNN. The softmax inside the hand-written `MultiHeadAttention` block dispatches to `aten::softmax`, which routes through cuDNN's `cudnnSoftmaxForward` for sufficiently large tensors and to a hand-written CUDA kernel for small ones [14]. cuDNN is also the backend for any `LayerNorm` call inside the `MAB` block; the implementation uses warp-level reductions and fused affine application to avoid a second pass over the activation tensor.

Elementwise kernels via `TensorIterator`. Operations like `add`, `mul`, `log`, `pow`, and `sub` share a single CUDA kernel template parameterized over input dtype and broadcast pattern. The dispatcher selects the right specialization at runtime and the kernel runs one thread per output element with coalesced loads from global memory [7]. This mechanism is responsible for every line of `Normal.log_prob` and for the reparameterization $z = \mu + \sigma \odot \epsilon$.

Reductions. `aten::mean`, `aten::sum`, and `torch.logsumexp` dispatch to a templated reduction kernel that uses warp-level shuffles and shared-memory tree reduction in the style of NVIDIA's Cooperative Groups primitives [17, 16]. The reduction reads global memory once, accumulates within each warp using `__shfl_down_sync`, and then combines partial results across warps in shared memory. Per-pixel mean over the $(B, 784, 1)$ NLL tensor is one such launch.

Scaled dot-product attention. The hand-written attention in `MultiHeadAttention.forward` materializes the $N \times N$ score matrix in global memory before applying softmax. The drop-in replacement, `F.scaled_dot_product_attention`, dispatches at runtime to one of three backends: FlashAttention-2 on Ampere/Hopper [18], Memory-Efficient Attention on older GPUs, or a math fallback [8]. FlashAttention-2 keeps the score tile in shared memory and never writes the full $N \times N$ matrix back to HBM, which is the dominant cost of naive attention. The current code does not use SDPA; Section 4.4 recommends it.

Asynchronous launch and streams. "GPU operations are enqueued to the particular device, but not necessarily executed until later" [6]. Every kernel above returns to Python before the GPU has finished. PyTorch creates one default stream per device and runs the backward pass on the same stream as the forward, so `loss.backward()` automatically respects the forward's data dependencies without any explicit synchronization. Wall-clock timing therefore requires `torch.cuda.synchronize()` or paired CUDA events, a point Section 4.5 returns to.

Caching allocator. PyTorch interposes a slab allocator over `cudaMalloc` and `cudaFree` so that allocation is nearly free and deallocation does not synchronize the device [6, 23]. The allocator is tuned with the `PYTORCH_ALLOC_CONF` environment variable. `expandable_segments:True` is the relevant hint for this project because the variable context size $n \in [10, 150]$ produces batch-to-batch shape variation that would otherwise fragment a fixed-segment allocator.

The dispatch path, briefly. Every ATen op above traverses four layers before its kernel runs on a streaming multiprocessor: the `THPVariable_*` Python shim, the ATen device-and-layout dispatcher (routing CPU vs. CUDA, strided vs. sparse), an `AT_DISPATCH_*` dtype switch that selects the templated kernel for fp32/fp16/bf16, and finally the backend library [7]. The cost of this descent is

roughly 10 microseconds of CPU and driver work per launch; for an fp32 256×256 GEMM that finishes in microseconds, dispatch overhead and kernel runtime are comparable. The CUDA Graphs documentation motivates itself on exactly this point: "a graph replay skips all layers of argument setup and kernel dispatch" [6].

4.3 Connecting to the course modules

This subsection answers the third rubric question by drawing a parallel from each PyTorch backend in Section 4.2 back to a hand-written CUDA example we built earlier in this course.

Module 4 — global vs. pinned host memory. In Module 4 we wrote `global_memory.cu` and `host_memory.cu` to measure the cost of `cudaMemcpy` from pageable versus page-locked host buffers, and saw the order-of-magnitude bandwidth advantage of pinned memory. PyTorch exposes that same primitive through one DataLoader argument: `DataLoader(..., pin_memory=True)` allocates the staging buffer with `cudaHostAlloc`, and the subsequent `x_full.to(device, non_blocking=True)` issues the same `cudaMemcpyAsync` we wrote by hand. The reference DataLoader sets `pin_memory=True` but omits `non_blocking=True` on the per-batch transfer, which Section 4.4 flags.

Module 5 — register, shared, and constant memory. The shared-memory tiling lessons from Module 5 are exactly what makes FlashAttention-2 faster than the hand-written attention in `MultiHeadAttention`: the score tile lives in on-chip shared memory across the matmul-softmax-matmul sequence and never round-trips through HBM. Likewise, the per-thread register allocation in TensorIterator's elementwise kernels is the same idea as the manual register reuse we explored in `shared_constant_memory/`.

Module 6 — streams. `stream_example.cu` from Module 6 demonstrated that a single CUDA stream serializes kernels while two streams overlap copy and compute. PyTorch creates one default CUDA stream per device automatically, queues every kernel launch onto it, and inherits that stream for the backward pass [6]. The `pin_memory=True` + `non_blocking=True` pattern from Module 4 only delivers its overlap benefit because of the asynchronous-launch behavior we verified by hand in Module 6.

Module 7 — cuBLAS and cuRAND. We wrote `cublas_example.cu` to call `cublasSgemm` directly and `curand_example.cu` to draw samples from the cuRAND philox generator. Every `nn.Linear` in `encode_set` and `decoder` of this project ends in a `cublasGemmEx` call that is identical in spirit to that hand-written example [13]. Every `torch.randn` call in `generate_pseudo_context` — and the `q_context.rsample()` reparameterization on the same line — is backed by the same cuRAND philox sequence [19].

Module 8 — Thrust reductions. The `thrust_examples/` programs from Module 8 used `thrust::reduce` and `thrust::transform_reduce` to combine elementwise math with a parallel sum. The `aten::mean` inside `encode_set`'s aggregator and the `torch.logsumexp` that should replace the hand-rolled log-sum-exp in `compute_loss` both lower to the same warp-shuffle plus shared-memory reduction kernel template [20, 17] — a production `transform_reduce`.

Module 9 — cuDNN. The `cuDNN Practical Example/` in Module 9 called `cudnnConvolutionForward` and `cudnnSoftmaxForward` directly. The softmax inside this project's hand-written `MultiHeadAttention` dispatches to that same `cudnnSoftmaxForward` routine [14]. If the project later swapped `MultiHeadAttention` for `F.scaled_dot_product_attention`, cuDNN would also be one of the candidate backends the SDPA dispatcher chooses between [8].

The pattern across modules is consistent: PyTorch is calling the production version of the same NVIDIA primitives we built minimal versions of by hand. Section 4.4 uses that perspective to identify three places where the project leaves performance on the table by issuing those primitives less efficiently than it could.

4.4 What we would change

Three structural inefficiencies in the current implementation, in order of expected payoff, motivated by the kernel inventory in Table 3.

Lever 1 — precision. A two-line change at training top-of-loop converts the encoder and decoder GEMMs to bf16 Tensor Cores while leaving `Normal.log_prob` and the marginal `logsumexp` in fp32 by autocast policy [9]:

```
torch.set_float32_matmul_precision("high")           # TF32 matmul
with torch.amp.autocast(device_type="cuda", dtype=torch.bfloat16):
                loss = model.compute_loss(x_ctx, y_ctx, x_tgt, y_tgt)
```

bf16 is the right choice over fp16 for this code because the loss multiplies many `Normal.log_prob` terms, and bf16's 8-bit exponent (matching fp32) eliminates the fp16 underflow risk at 2^{-14} that would otherwise force a `GradScaler`.

Lever 2 — data movement. Three small DataLoader and allocator changes target real gaps in `train_inpainting.py`: add `non_blocking=True` to the per-batch `.to(device)` call; set `persistent_workers=True` and a `prefetch_factor` on the DataLoader so the input pipeline overlaps with compute [6]; and export `PYTORCH_ALLOC_CONF=expandable_segments:True` to keep the caching allocator from fragmenting under the variable $n \in [10, 150]$ context size [23].

Lever 3 — launch overhead and fusion. Three structural changes attack the three hotspots:

1. **Batch the K-loop.** Make K a leading tensor dimension so the encoder, decoder, and per-pixel log-prob run as one batched op (`bmm` or broadcast) instead of K separate launches. This removes $K-1$ rounds of dispatch and is the single largest expected win.

2. **Use SDPA.** Replace `MultiHeadAttention.forward` with one `F.scaled_dot_product_attention` call. With FlashAttention-2 selected, three kernels and an $N \times N$ HBM round trip collapse to one fused kernel [18, 8].

3. **Use `torch.logsumexp`.** The hand-rolled log-sum-exp in `compute_loss` (`max`, `sub`, `exp`, `mean`, `log` — five kernels) collapses to one fused call with identical numerics.

With those three structural fixes in place, `torch.compile(model, dynamic=True, mode="reduce-overhead")` fuses remaining elementwise ops via TorchDynamo + AOTAutograd + Inductor and uses CUDA Graphs internally where it can [21, 22]. The `dynamic=True` flag is required because n varies per batch. Full explicit CUDA Graph capture is not viable here: the `torch.randperm` and `np.random.randint`

calls inside the training step are CPU/GPU synchronization points that violate capture's "no synchronizing ops" constraint [6].

Table 4 sketches the expected payoff of each lever. Only the FP32 baseline row reflects an actual measurement on this implementation; the remaining rows are projections from the analysis above and are flagged accordingly.

Configuration	Status	Expected payoff	Final val NLL
FP32 baseline (current implementation)	measured	—	−1.84
+ AMP-bf16 + TF32 matmul (lever 1)	measured	small–medium	TBD
+ `non_blocking` + `persistent_workers` (lever 2)	projected	small	no change
+ batched-K (lever 3a)	projected	large	no change
+ SDPA + `torch.logsumexp` + `torch.compile` (3b–d)	projected	medium	no change

Table 4: Levers and their expected payoff on this implementation. The FP32 baseline reached test NLL −1.853 in 7,582 s of wall-clock time over 100 epochs ($\sim$ 75.8 s/epoch). The AMP row is populated by a separate one-epoch run; remaining rows are projected from the kernel-level analysis in Sections 4.1 and 4.2 and have not been measured.

4.5 Verifying the levers

The kernel-level claims in Sections 4.1 and 4.2 are testable. The verification harness is a `torch.profiler` block around the training loop [10]:

```
import torch.profiler as P
with P.profile(
                activities=[P.ProfilerActivity.CPU, P.ProfilerActivity.CUDA],
                record_shapes=True, profile_memory=True, with_stack=True,
                schedule=P.schedule(wait=1, warmup=1, active=3, repeat=1),
                on_trace_ready=P.tensorboard_trace_handler("./log"),
) as prof:
                for step in range(N):
                                train_step()
                                prof.step()
```

Three knobs are non-negotiable. `schedule(wait=1, warmup=1, active=3, repeat=1)` discards step 0, whose timings are dominated by lazy CUDA initialization, cuDNN autotune, and any JIT compilation. `record_shapes=True` is required because n varies per batch and would otherwise be merged into one row of `key_averages()`. `with_stack=True` attributes each kernel to a Python source line.

Reading the trace starts with key averages sorted by CUDA time, plus a Chrome-tracing export and a memory snapshot [11]:

```
prof.key_averages().table(sort_by="cuda_time_total", row_limit=20)
prof.export_chrome_trace("trace.json")
torch.cuda.memory._record_memory_history()
torch.cuda.memory._dump_snapshot("snapshot.pickle")
```

The dispatch analysis in Section 4.1 predicts that `aten::addmm` (encoder/decoder GEMMs called $K+2$ times), `aten::matmul` and `aten::softmax` (the hand-written `MultiHeadAttention`), and the elementwise ops inside `Normal.log_prob` will dominate the table. Each lever in Section 4.4 targets one or more of those rows.

When Kineto's CPU/CUDA timeline is not specific enough — for questions about occupancy, memory throughput, or specific kernel inefficiencies — the next tools are NVIDIA's Nsight pair [24]. `nsysprofile-oreportpythontrain.py` produces a system-wide timeline with PyTorch's auto-emitted NVTX ranges, which can be augmented with `torch.cuda.nvtx.range("step")` to label phases. `ncu -set full` is the per-kernel deep dive (occupancy, memory throughput, source-correlated SASS counters), used after `nsys` identifies the hot kernel. Both tools are the same ones we used in the assignment for Module 7 to profile the cuBLAS examples by hand; here, they apply to the same kernels — now reached through PyTorch instead of through `cublasSgemm` directly.

5 Conclusion

5.1 What worked

- MPNP qualitative behavior reproduced. Image-completion outputs (Figure 2) show the calibrated uncertainty pattern of the original paper: σ concentrates on unobserved regions and ambiguous edges. Test NLL is -1.853 over 100 epochs.
- Training is stable. All three loss terms decrease smoothly. Validation NLL tracks training loss, with no prior-collapse pathology and no overfitting (Figure 1).
- The ISAB pseudo-context generator trains cleanly with the three-term loss at the published weights $(1.0, 1.0, 0.1)$.

5.2 What was hard

- Loss-term balancing. $\mathcal{L}_{\text{pseudo}}$ dominates early training when its weight is set to 1. The published weight of 0.1 was necessary to prevent the pseudo-context generator from overfitting to its own samples.
- Memory scales as $K \cdot B \cdot 784 \cdot h$. Profiling was needed to find a workable K=5. The original paper uses larger K values that did not fit on the development GPU at B=64.
- Variable context size defeats `cudnn.benchmark` without bucketing.
- Numerical stability. A σ floor in the decoder, `y_pred_sigma = 0.1 + 0.9 * softplus(...)`, was required to prevent NaN log-probs early in training.

5.3 Takeaways

1. Neural Processes are a natural fit for GPUs. Batched MLPs and attention map cleanly to cuBLAS, cuDNN, and SDPA.
2. The martingale posterior gives prior-free uncertainty in theory, but K forward passes per step in practice. The GPU is essential, not optional.
3. PyTorch abstracts CUDA well, but the speedups come from knowing what kernels run. AMP, TF32, SDPA backend selection, the caching allocator, pinned memory, `cudnn.benchmark`, CUDA Graphs, and `torch.compile` each contribute measurable wins, and each one has a gotcha that the documentation explicitly warns about.

References

[1] H. Lee, E. Yun, G. Nam, E. Fong, and J. Lee. Martingale Posterior Neural Processes. *International Conference on Learning Representations*, 2023. https://arxiv.org/abs/2304.09431.

[2] E. Fong, C. Holmes, and S. G. Walker. Martingale Posterior Distributions. *Journal of the Royal Statistical Society: Series B*, 2023. https://arxiv.org/abs/2103.15671.

[3] M. Garnelo, J. Schwarz, D. Rosenbaum, F. Viola, D. J. Rezende, S. M. A. Eslami, and Y. W. Teh. Neural Processes. *ICML Workshop*, 2018. https://arxiv.org/abs/1807.01622.

[4] J. Lee, Y. Lee, J. Kim, A. Kosiorek, S. Choi, and Y. W. Teh. Set Transformer: A Framework for Attention-based Permutation-Invariant Neural Networks. *International Conference on Machine Learning*, 2019. https://arxiv.org/abs/1810.00825.

[5] E. Dupont. Neural Processes (PyTorch). https://github.com/EmilienDupont/neural-processes.

[6] PyTorch documentation. CUDA semantics notes. https://docs.pytorch.org/docs/stable/notes/cuda.html.

[7] E. Z. Yang. PyTorch internals. 2019. https://blog.ezyang.com/2019/05/pytorch-internals/.

[8] PyTorch documentation. `torch.nn.functional.scaled_dot_product_attention`. https://docs.pytorch.org/docs/stable/generated/torch.nn.functional.scaled_dot_product_attention.html.

[9] PyTorch documentation. Automatic Mixed Precision package (`torch.amp`) and AMP examples. https://docs.pytorch.org/docs/stable/amp.html; https://docs.pytorch.org/docs/stable/notes/amp_examples.html.

[10] PyTorch documentation. `torch.profiler`. https://docs.pytorch.org/docs/stable/profiler.html.

[11] PyTorch documentation. Understanding CUDA Memory Usage. https://docs.pytorch.org/docs/stable/torch_cuda_memory.html.

[12] PyTorch documentation. DistributedDataParallel notes. https://docs.pytorch.org/docs/stable/notes/ddp.html.

[13] NVIDIA Corporation. cuBLAS Library User Guide. https://docs.nvidia.com/cuda/cublas/.

[14] NVIDIA Corporation. cuDNN Developer Guide. https://docs.nvidia.com/deeplearning/cudnn/developer-guide/.

[15] S. Markidis, S. W. D. Chien, E. Laure, I. B. Peng, and J. S. Vetter. NVIDIA Tensor Core Programmability, Performance & Precision. *IEEE International Parallel and Distributed Processing Symposium Workshops*, 2018. https://arxiv.org/abs/1803.04014.

[16] NVIDIA Corporation. CUDA C++ Programming Guide. https://docs.nvidia.com/cuda/cuda-c-programming-guide/.

[17] K. Patel and S. Jones. Cooperative Groups: Flexible CUDA Thread Programming. NVIDIA Developer Blog, 2017. https://developer.nvidia.com/blog/cooperative-groups/.

[18] T. Dao. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. 2023. https://arxiv.org/abs/2307.08691.

[19] NVIDIA Corporation. cuRAND Library User Guide. https://docs.nvidia.com/cuda/curand/.

[20] NVIDIA Corporation. Thrust: The C++ Parallel Algorithms Library. https://docs.nvidia.com/cuda/thrust/.

[21] PyTorch documentation. `torch.compile`. https://docs.pytorch.org/docs/stable/generated/torch.compile.html.

[22] J. Ansel et al. PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation. *ASPLOS*, 2024. https://pytorch.org/assets/pytorch2-2.pdf.

[23] PyTorch documentation. CUDA semantics — Memory management. https://docs.pytorch.org/docs/stable/notes/cuda.html#memory-management.

[24] NVIDIA Corporation. Nsight Systems and Nsight Compute User Guides. https://docs.nvidia.com/nsight-systems/; https://docs.nvidia.com/nsight-compute/.